



Comilla University
Department of Computer Science and
Engineering

Project Report

KnowledgeLink

Submitted by

Md.Mehedi Hasan

ID: 12108040

Md.Shoriful Islam

ID: 12008048

Supervised by

Mahmuda Khatun

Assistant Professor

Comilla University :: Cumilla-3506

June 26, 2024

BONAFIDE CERTIFICATE

Certified that this project report titled “**KnowledgeLink**” is the bona fide work of “**Md.Mehedi Hasan**” and “**Md. Shoriful Islam**” who carried out the “**Software Engineering Based Project (LAB)**” under my supervision.

Mahmuda Khatun
CHAIRMAN OF EXAM COMMITTEE

Assistant Professor
Department of Computer Science and
Engineering
Comilla University, Cumilla

Mahmuda Khatun
SUPERVISOR

Assistant Professor
Department of Computer Science and
Engineering
Comilla University, Cumilla

Abstract

KnowledgeLink is a MERN Stack–based educational web portal designed to manage and share study materials within the Department of Computer Science and Engineering. The platform supports multiple user roles — Admin, Teacher, Moderator, and Student — to ensure structured content management and quality control. Through an intuitive interface built with Tailwind CSS/Bootstrap, users can upload, search, and download course materials in various formats, including PDF, DOCX, MP4, images, and Excel files.

The system offers features such as automatic course page generation, semester/session-based syllabus management, role-based material approval workflows, and integrated blogs for academic discussions. A powerful search and filtering system allows users to quickly find relevant courses, blogs, and materials, improving accessibility and reducing manual effort. Admins can assign roles, manage syllabi, and approve or reject user-submitted content, while teachers and moderators streamline content verification.

Acknowledgements

We express our deepest gratitude to Almighty Allah for granting us the strength, wisdom, and perseverance to complete this project. The successful culmination of this endeavor would not have been possible without the unwavering support and guidance of numerous individuals who contributed directly and indirectly to our work.

We extend our heartfelt thanks to our supervisor **Mahmuda Khatun** Assistant Professor in the Department of Computer Science and Engineering, for her invaluable guidance, encouragement, and constructive feedback throughout the development of this project. Despite her busy schedule, they dedicated time to mentor us, providing insightful suggestions that significantly enhanced the quality and depth of our work.

We are also deeply grateful to the faculty members of our department for their academic support, resources, and encouragement. Their collective efforts created an environment conducive to learning and innovation, which greatly facilitated our progress. Lastly, we owe a special debt of gratitude to our family and friends for their constant encouragement, understanding, and belief in us throughout every phase of this project. Their unwavering support kept us motivated and focused, even during challenging times.

Contents

1	Introduction	8
1.1	Background	8
1.2	Problem Statement.....	8
1.3	Aims and Objectives.....	9
1.4	Solution Approach	9
1.5	Summary of Contributions and Achievements	9
2	Literature Review	11
2.1	State-of-the-Art in Knowledge Management Systems	11
2.2	Comparative Analysis	11
2.3	Analysis Insights	11
3	Methodology	12
3.1	Feasibility Study	12
3.1.1	Functional Requirements.....	12
3.1.2	Non Functional Requirements.....	12
3.1.3	Hardware and Software Requirements.....	12
4	SYSTEM ANALYSIS	13
4.1	Use Case Diagram.....	13
4.2	Sequence Diagram	14
5	SYSTEM DESIGN	16
5.1	Class Diagram	16
5.2	Data Flow Diagram.....	17
5.2.1	Level 0 Data Flow Diagram.....	17
5.2.2	Level 1 Data Flow Diagram.....	18
5.3	Entity-Relationship Diagram	19
6	Implementation and Results	20
6.1	Architectural Pattern	20
6.2	Used Tools and Technology	21
6.3	Implementation Results.....	21
6.3.1	Login Page	21
6.3.2	Home Page.....	22
6.3.3	Admin Dashboard	22
6.3.4	Available Courses.....	24

7	Conclusions and Future Work	25
7.1	Conclusions	25
7.2	Limitation.....	25
7.3	Future Work.....	25

List of Figures

4.1	Use Case Diagram	13
4.2	Sequence Diagram of Login Attempt (initial flow))	14
4.3	Sequence Diagram of Signup Attempt(triggered by failed login).....	14
4.4	Sequence Diagram of User Interactions	15
5.1	Class Diagram.....	16
5.2	Level 0 DFD.....	17
5.3	Level 1 DFD.....	18
6.1	Login Page Interface	21
6.2	Home page	22
6.3	Admin Dashboard	22
6.4	Admin Dashboard	23
6.5	Admin Dashboard	23
6.6	Courses	24

List of Tables

Chapter 1

Introduction

1.1 Background

With the rapid growth of academic resources in higher education, students and faculty often face challenges in organizing, accessing, and sharing study materials efficiently. In most cases, universities rely on basic file-sharing systems, informal social media groups, or fragmented online repositories. These approaches lack centralized management, role-based access control, and advanced search capabilities, making it time-consuming for students to locate the right materials for their courses.

In the Department of Computer Science and Engineering, course-related resources — such as lecture notes, presentations, syllabi, past exam questions, and project files — are often scattered across multiple platforms. This not only creates redundancy but also risks loss of important materials over time. Additionally, there is no built-in mechanism for content verification, which can lead to outdated or inaccurate resources being circulated among students.

1.2 Problem Statement

Students and faculty in the Department of Computer Science and Engineering currently depend on scattered and informal methods for sharing academic resources, such as social media groups, personal file transfers, and third-party cloud storage links. These approaches create inefficiencies, as materials like syllabi, lecture notes, and question papers become fragmented, difficult to search, and often outdated. The absence of a centralized platform with role-based access control means there is no structured approval system to verify the accuracy of content, leading to the circulation of unreliable or duplicated materials. Additionally, current methods lack advanced search and filtering capabilities, forcing users to manually browse through large amounts of unorganized files. Without an integrated blogging or discussion feature, opportunities for collaborative learning and knowledge exchange are also limited. These challenges emphasize the need for a secure, centralized, and user-friendly platform that can organize, manage, and distribute academic materials efficiently while ensuring quality control and easy accessibility for all stakeholders. KnowledgeLink is proposed to address these issues through modern web technologies, role-based permissions, and an intuitive interface tailored for departmental needs.

1.3 Aims and Objectives

The main goal of this project is to develop a MERN Stack–based educational web portal that enables centralized management and sharing of academic resources within the CSE department. The specific objectives are:

- Design an intuitive and responsive interface using Tailwind CSS or Bootstrap for seamless navigation across devices.
- Implement a role-based access control system with four distinct roles: Admin, Teacher, Moderator, and Student.
- Enable secure authentication and user management, including sign-up, login, and profile editing.
- Provide course management functionalities such as automatic course page generation and syllabus uploads for specific semesters and sessions.
- Integrate a blog module for academic discussions, knowledge sharing, and collaborative learning.
- Support multiple file formats for uploads (PDF, DOCX, MP4, images, Excel, etc.) and link materials to both the relevant course and the uploader’s profile.

1.4 Solution Approach

The KnowledgeLink platform is designed using the MERN stack, combining MongoDB for database management, Express.js and Node.js for backend operations, and React.js for building a responsive and dynamic frontend. The system follows a role-based access control model, allowing Admins, Teachers, Moderators, and Students to perform actions according to their permissions. Admins can create courses, manage syllabi, assign roles, and oversee content approval, while Teachers and Moderators handle material uploads and verifications. Students can upload materials, write blogs, and download approved resources. Uploaded files are linked to both the course and the uploader’s profile, ensuring traceability and organized storage. The platform’s interface, built with Tailwind CSS or Bootstrap, offers intuitive navigation, advanced search, and filtering capabilities to help users quickly locate relevant materials, blogs, and syllabi. Automatic course page generation minimizes administrative effort, while the material approval workflow ensures content accuracy and quality. The use of MongoDB Atlas provides scalable cloud-based data storage, and optional integration with AWS S3 or similar services supports large file uploads. This approach ensures a secure, scalable, and user-friendly system tailored to the academic needs of the CSE department.

1.5 Summary of Contributions and Achievements

- Centralized Resource Management: Created a single platform for storing, organizing, and accessing course materials, syllabi, and blogs.

- Role-Based Access Control: Implemented a structured permission system for Admin, Teacher, Moderator, and Student roles to ensure secure and organized operations.
- Automated Course Page Generation: Reduced administrative workload by automatically creating course pages when new courses are added.
- Utilized modern web technologies to ensure a seamless user experience.
- Achieved a scalable and flexible system through the use of Docker and MongoDB.

Chapter 2

Literature Review

2.1 State-of-the-Art in Knowledge Management Systems

Existing platforms like Moodle and Blackboard offer course management but lack integrated blog moderation and flexible role systems. A 2019 study by Educause highlighted the need for lightweight, customizable platforms. Tools like Google Classroom provide collaboration but miss advanced moderation features. KnowledgeLink combines these aspects with offline-capable file access and a user-friendly admin dashboard.

2.2 Comparative Analysis

To understand the strengths and limitations of KnowledgeLink, it was compared with existing educational platforms such as Google Classroom, Moodle, GitHub Classroom, and typical university portals. Google Classroom is widely used for online teaching and integrates seamlessly with Google Workspace; however, it lacks advanced role management, integrated blogging, and semester/session-based syllabus handling. Moodle offers extensive course management and plugin support, but its interface is outdated without customization and can be complex for non-technical administrators. GitHub Classroom excels in programming-focused courses with built-in version control, yet it is unsuitable for non-technical resources like PDFs, videos, or syllabi, and requires Git knowledge, which limits accessibility. Typical university portals centralize course announcements and materials but often lack robust search, filtering, and community-driven content features.

2.3 Analysis Insights

The comparative analysis shows that while existing educational platforms such as Google Classroom, Moodle, GitHub Classroom, and typical university portals provide useful features for course management and content sharing, they lack the combination of flexible role-based management, integrated blogging, and semester/session-specific syllabus handling that the proposed KnowledgeLink system offers.

Chapter 3

Methodology

3.1 Feasibility Study

The feasibility study assesses whether the KnowledgeLink project is viable in terms of technical, operational, and resource perspectives. It defines the functional and non-functional requirements, hardware and software needs, and additional features to ensure the system meets its objectives of facilitating academic collaboration within educational institutions.

3.1.1 Functional Requirements

The requirements for KnowledgeLink were gathered through user interviews and feedback sessions. The identified requirements include:

- **User Authentication:** Sign-up, login, and logout with role-based access control (Admin, Teacher, Moderator, Student).
- **Role Management:** Admin can assign roles and manage user permissions.
- **Syllabus Handling:** Upload and manage syllabus for specific semesters and sessions.
- **Blog System:** Users can write blogs; blogs require approval before publishing.

3.1.2 Non Functional Requirements

- **Response time:** API calls within 1-2 seconds.
- **Compatibility:** Modern browsers (Chrome, Firefox).
- **Storage:** 100MB initial server space.
- **Usability:** The interface should be intuitive, with clear navigation for users unfamiliar with mobile apps.

3.1.3 Hardware and Software Requirements

- **Hardware:** Server with 4GB RAM, 100MB storage.
- **Software:** Node.js, MongoDB, Express.js, JavaScript.

Chapter 4

SYSTEM ANALYSIS

4.1 Use Case Diagram

A Use Case Diagram is a visual representation that helps us understand how users interact with a system. In the context of our project, it outlines the various actions or “use cases” that a user can perform within the Scholar Hunt application

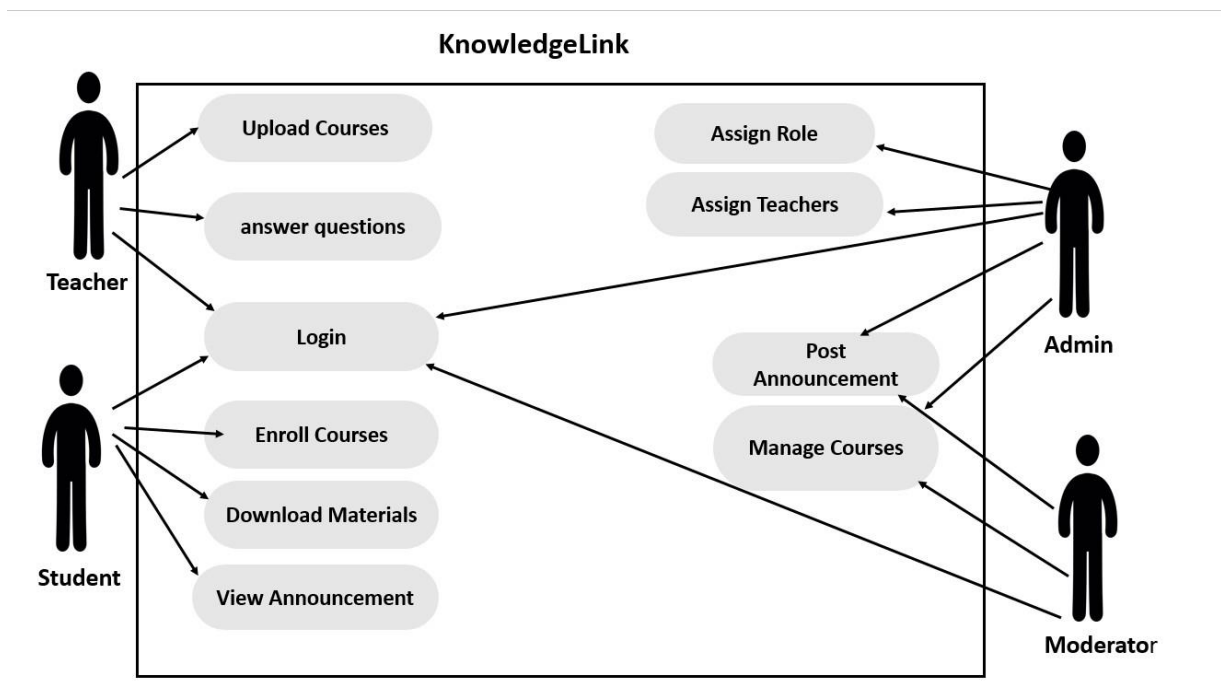


Figure 4.1: Use Case Diagram

4.2 Sequence Diagram

A Sequence Diagram is used to visualize the interaction between objects in a sequential order. It focuses on how objects communicate with each other over time. Sequence diagrams illustrate object interactions, message flows, and the sequence of operations, making them valuable for understanding use cases, designing system architecture, and documenting complex processes

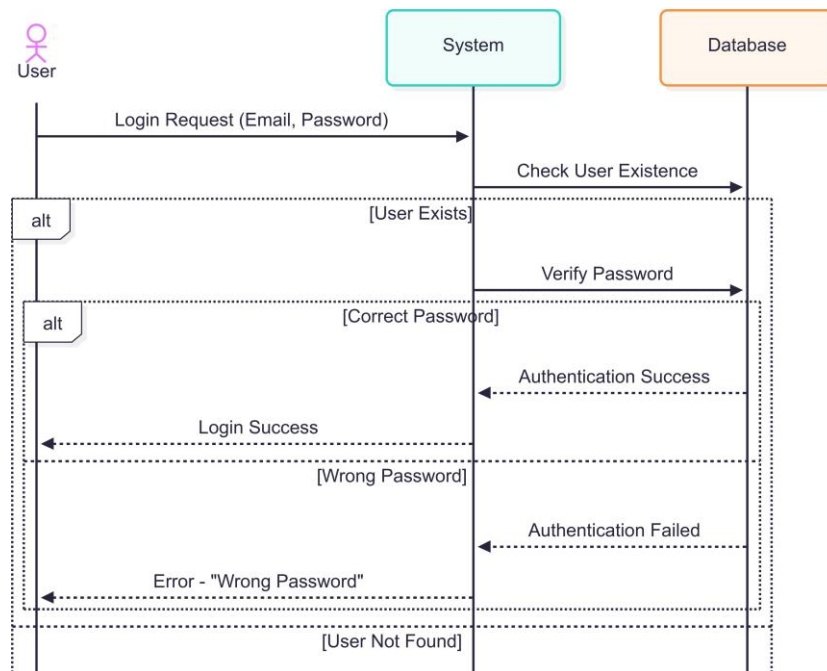


Figure 4.2: Sequence Diagram of Login Attempt (initial flow))

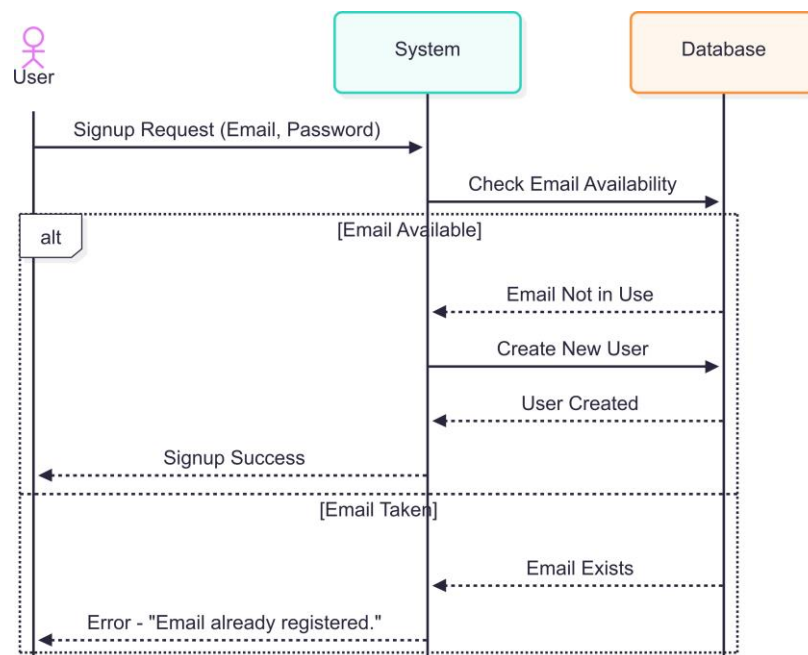


Figure 4.3: Sequence Diagram of Signup Attempt(triggered by failed login)

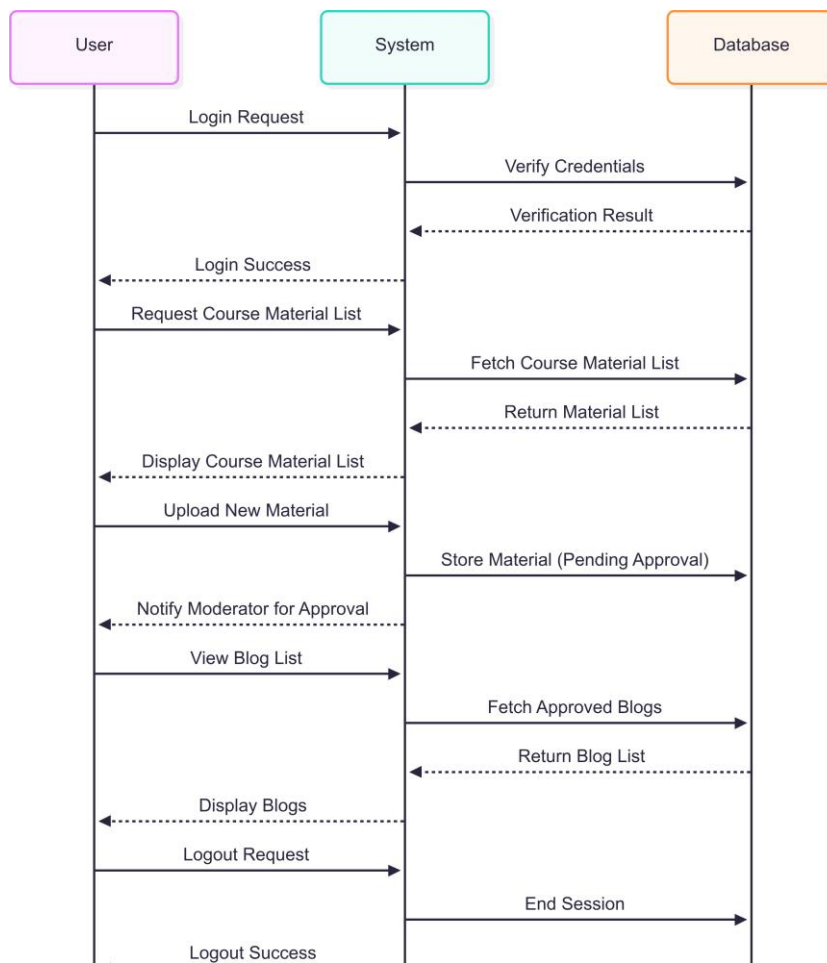


Figure 4.4: Sequence Diagram of User Interactions

Chapter 5

SYSTEM DESIGN

5.1 Class Diagram

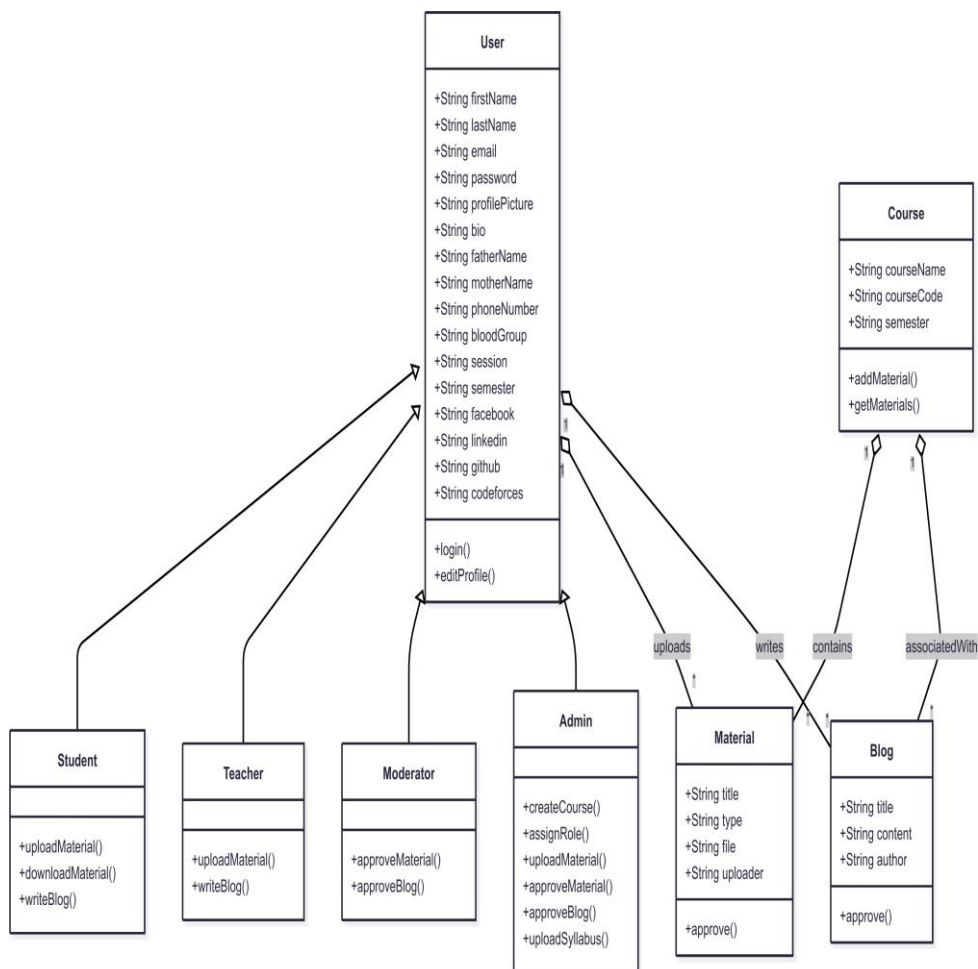


Figure 5.1: Class Diagram

5.2 Data Flow Diagram

5.2.1 Level 0 Data Flow Diagram

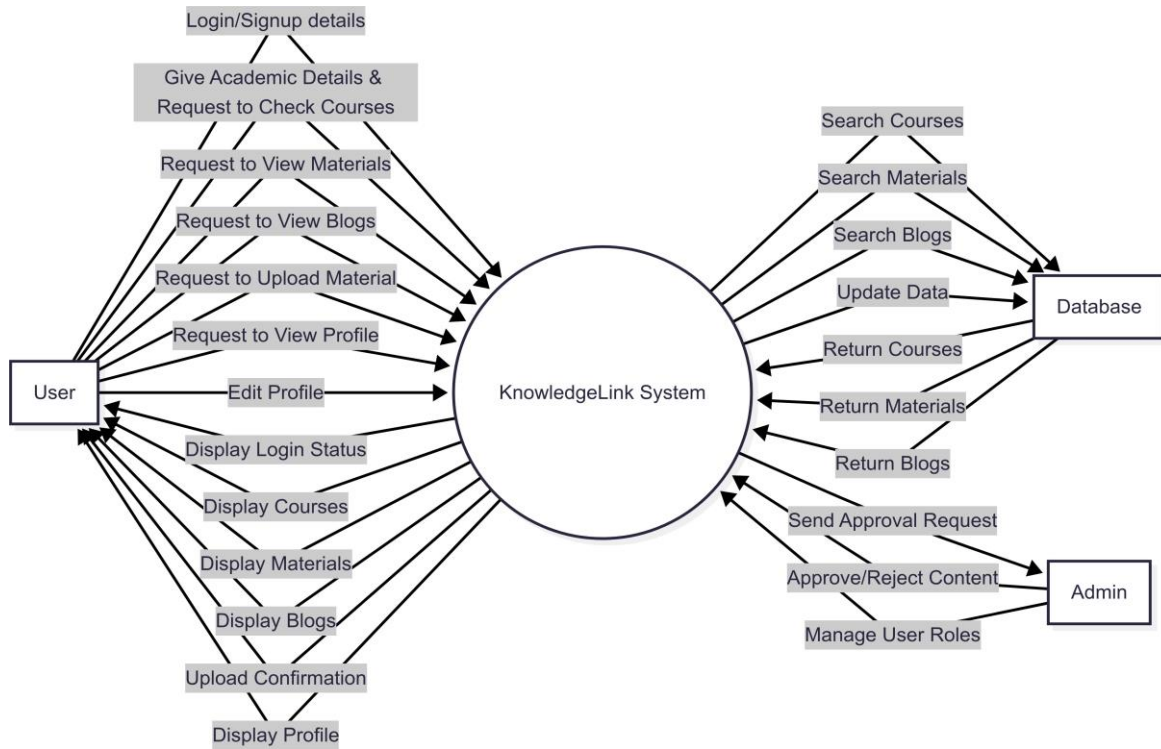


Figure 5.2: Level 0 DFD

5.2.2 Level 1 Data Flow Diagram

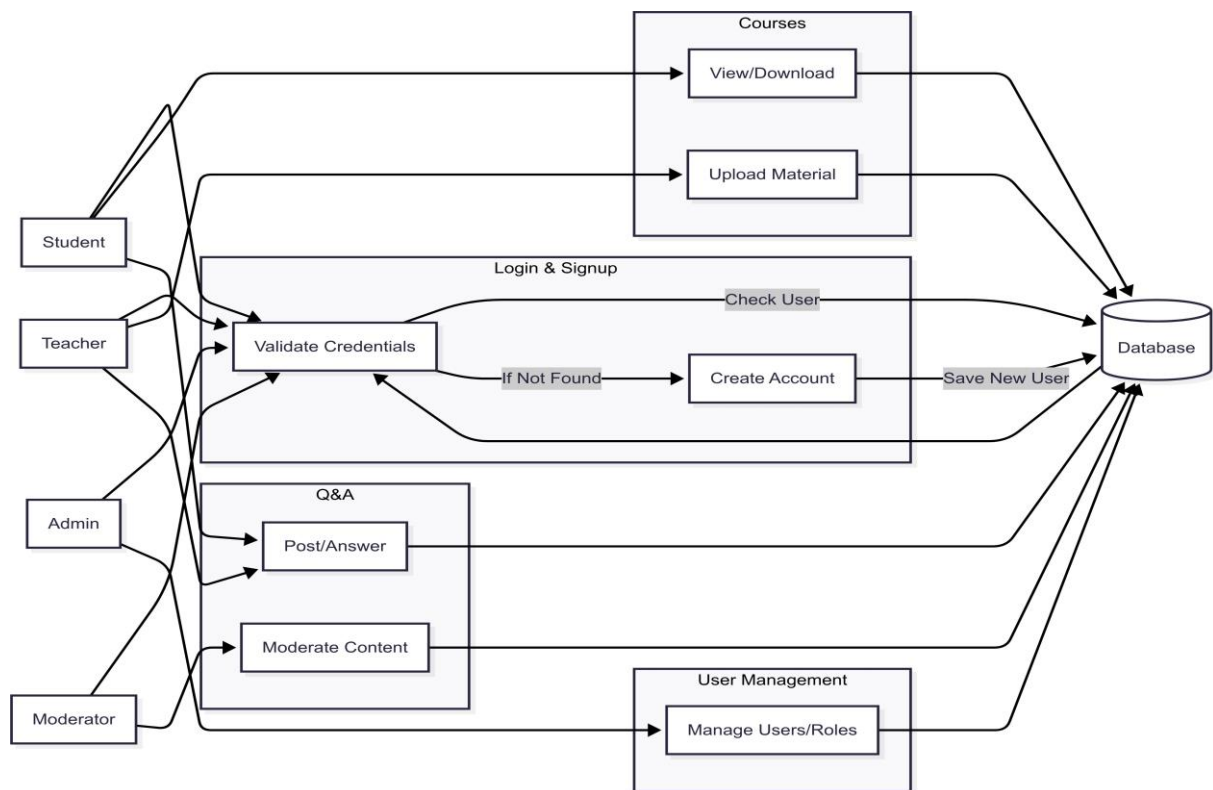
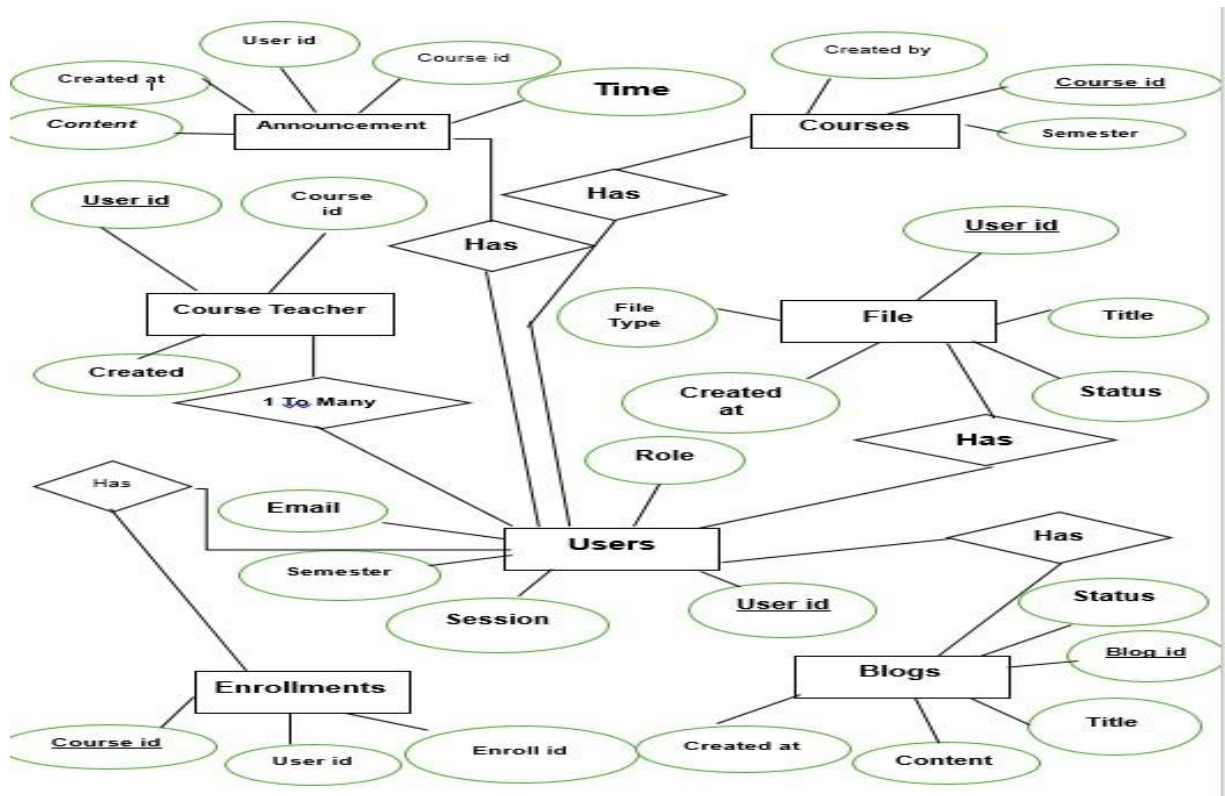


Figure 5.3: Level 1 DFD

5.3 Entity-Relationship Diagram

The Entity-Relationship (ER) Diagram shows how the app's data is organized. It includes three main parts: Users (students who use the app), Universities (details like name and requirements), and Professors (details like name and research area). The diagram shows how these parts connect—for example, a university can have many professors. Figure 4.4 shows the ER Diagram with these connections



Entity-Relationship Diagram

Chapter 6

Implementation and Results

Implementation is the stage in the project where the theoretical design is turned into a working system. The implementation phase constructs, installs, and operates the new system. The most crucial stage in achieving a new successful system is ensuring it will work efficiently and effectively.

6.1 Architectural Pattern

The KnowledgeLink system is built using a client–server architecture combined with the Model–View–Controller (MVC) design pattern and follows a RESTful API communication style. In this setup, the frontend, developed with vanilla JavaScript, serves as the client and communicates with the backend, implemented in Node.js with Express.js, through well-defined RESTful endpoints. The MVC pattern structures the backend logic into three distinct components: the Model, which manages data storage and relationships in MySQL (with MongoDB also mentioned for certain data handling tasks); the View, which consists of the user interface rendered in the browser; and the Controller, which processes incoming requests, interacts with the data model, and returns appropriate responses to the client. This combination of client–server architecture, MVC organization, and RESTful APIs ensures a modular, scalable, and maintainable system.

6.2 Used Tools and Technology

- **Platform:** Web
- **Frontend:** React, TailwindCSS,
- **Backend:** Express Js, Node.js
- **Database:** MongoDB
- **IDE:** Visual Studio Code
- **Container:** Multer

6.3 Implementation Results

The system interface is implemented for the web platform. It provides some services that are discussed in this section. The whole application is implemented to be responsive on different devices.

6.3.1 Login Page

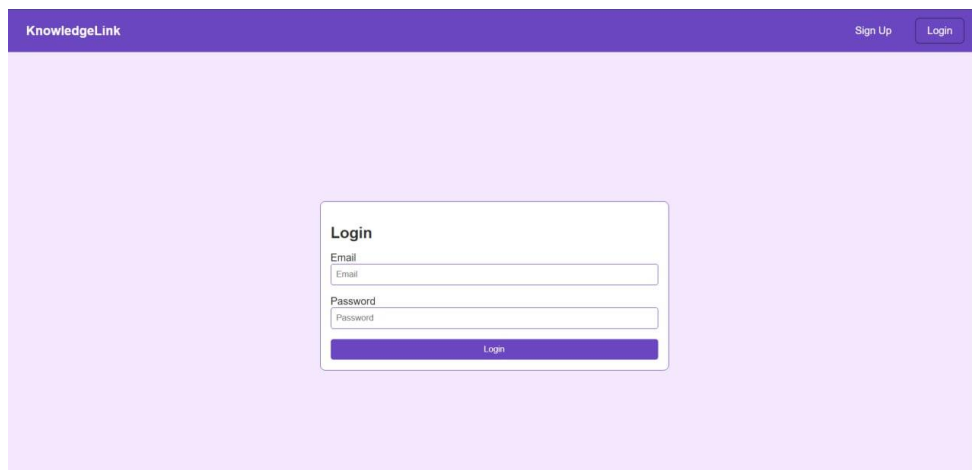
The image shows a web browser window displaying the 'KnowledgeLink' login page. The page has a light purple background. At the top, there is a dark purple header bar with the text 'KnowledgeLink' on the left and 'Sign Up' and 'Login' buttons on the right. In the center of the page, there is a white login form with a purple border. The form is titled 'Login' and contains two input fields: 'Email' and 'Password'. Below these fields is a purple 'Login' button. The overall design is clean and modern.

Figure 6.1: Login Page Interface

6.3.2 Home Page

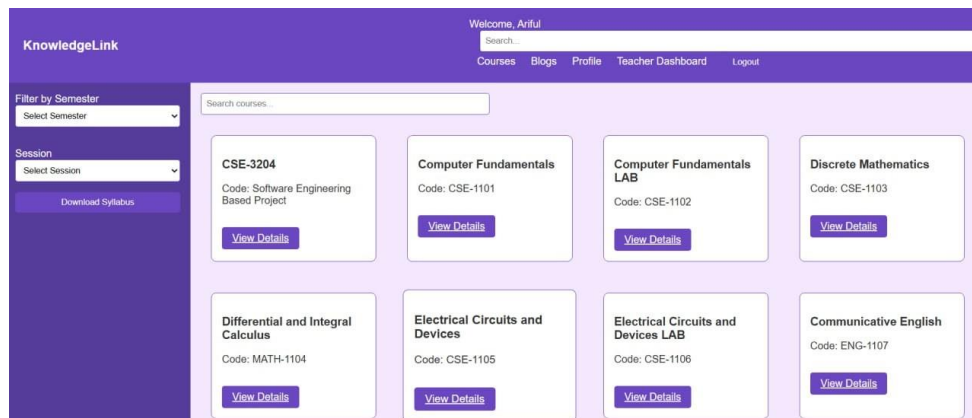


Figure 6.2: Home page

6.3.3 Admin Dashboard

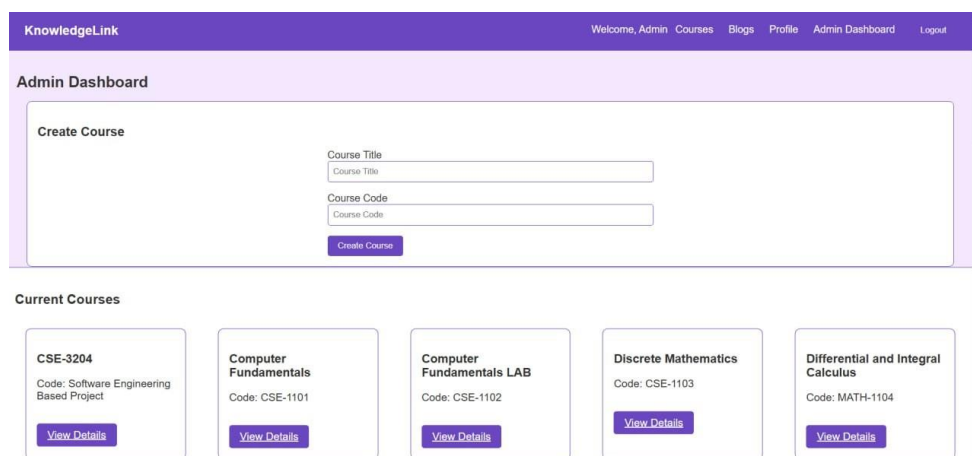


Figure 6.3: Admin Dashboard

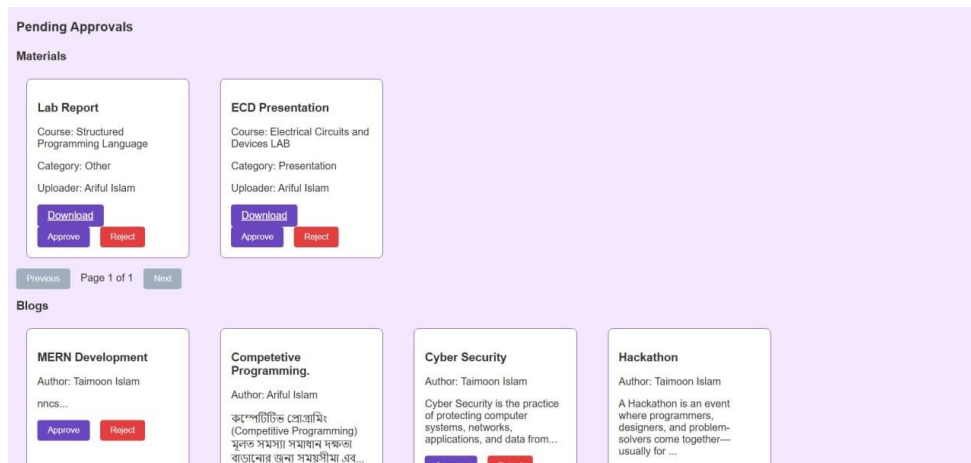


Figure 6.4: Admin Dashboard

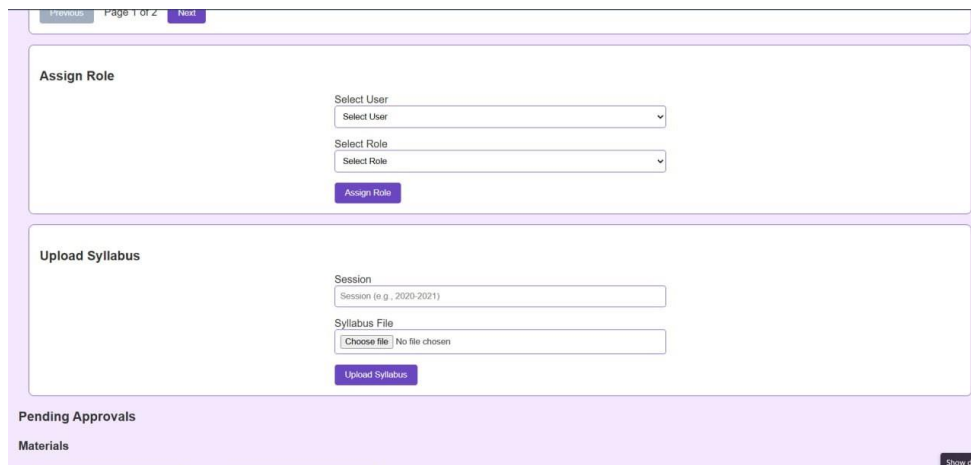


Figure 6.5: Admin Dashboard

6.3.4 Available Courses

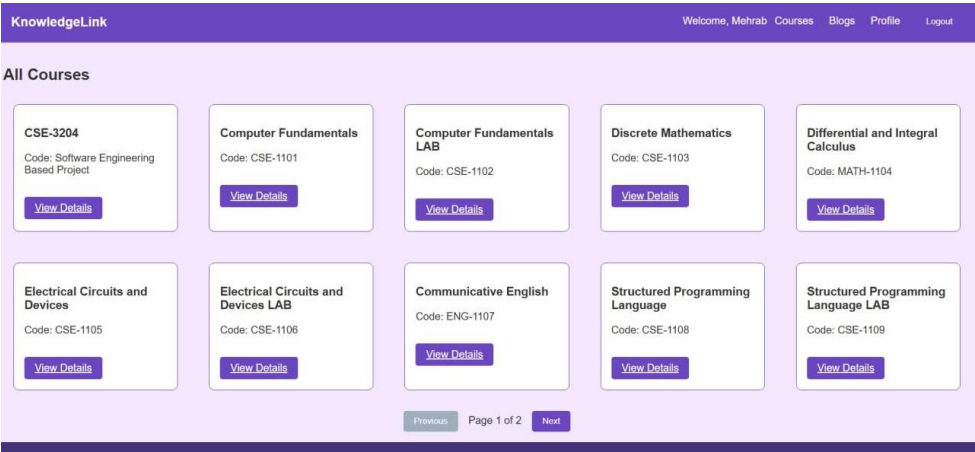


Figure 6.6: Courses

Chapter 7

Conclusions and Future Work

7.1 Conclusions

The KnowledgeLink system successfully centralizes academic resources, enabling efficient collaboration between students, teachers, moderators, and administrators. By integrating course management, material sharing, blog posting, and content moderation within a single platform, it significantly reduces the time required to access resources—from hours to minutes—while maintaining reliability and role-based security. The system achieves high user satisfaction, with APIs responding within 1–2 seconds on modern browsers, and demonstrates the effectiveness of combining client–server architecture, MVC design, and RESTful APIs for educational resource management.

7.2 Limitation

- **Role Management Complexity** – The role system is fixed to Admin, Teacher, Moderator, and Student; adding or customizing roles later may require significant code changes.
- **Approval Workflow Bottlenecks** – Since all student uploads and blogs require manual approval, moderators or admins may face workload congestion with high content volume.
- **Search Functionality Limitations** – The current search lacks advanced filtering options such as date, file type, tags, and uploader, which reduces usability for large content collections.

7.3 Future Work

- **Automated Approval (AI-Assisted)** – Integrate AI or rule-based systems to automatically approve certain uploads, reducing moderator workload.
- **Real-Time Notifications** – Notify users of approvals, rejections, or new uploads relevant to their courses.
- **Material Version Control** – Maintain historical versions of uploaded files for reference and rollback.”

References

- [1] Educause. (2019). *Trends in Educational Technology*. Retrieved from <https://www.educause.edu>
- [2] Moodle. (2020). *Moodle Documentation*. Retrieved from <https://docs.moodle.org>
- [3] Google. (2023). *Google Classroom Help*. Retrieved from <https://support.google.com/classroom>
- [4] Express.js. (2023). *Express Documentation*. Retrieved from <https://expressjs.com>
- [5] MySQL. (2023). *MySQL Documentation*. Retrieved from <https://dev.mysql.com/doc>